

INF 662 · DESARROLLO DE APLICACIONES MÓVILES

Guía de Laboratorio N.º 4

Listas, cuadrículas y navegación con paso de datos

MATERIA INF 662 — Desarrollo de Aplicaciones Móviles**SEMESTRE** 6.º · Mención Ing. de Software**TEMA** Listas, cuadrículas y datos entre pantallas**DURACIÓN** 5 horas (1 sesión de laboratorio)**CAPÍTULO** Cap. 2 — Explorando Widgets y navegación**MODALIDAD** Laboratorio · individual**DOCENTE** M. Sc. Lic. Huáscar Fedor Gonzales Guzmán**REQUISITO** Guías de Laboratorio N.º 1 a N.º 3**NOTA · Continúa desde la Guía N.º 3**

Ya sabes componer una pantalla y navegar entre dos. Ahora mostrarás **colecciones de datos** con listas y cuadrículas desplazables, y aprenderás a **pasar información** de una pantalla a otra y a recibir un resultado de vuelta.

01 Objetivos

Objetivo general. Mostrar colecciones de datos mediante listas y cuadrículas desplazables, y comunicar información entre pantallas al navegar, construyendo un pequeño catálogo interactivo.

Objetivos específicos:

- Modelar datos con una clase simple y una lista de objetos.
- Construir listas con `ListView` y `ListView.builder` usando `ListTile`.
- Presentar datos en cuadrícula con `GridView.builder` y `Card`.
- Pasar datos a otra pantalla por el constructor y `MaterialPageRoute`.
- Devolver un resultado con `Navigator.pop(context, valor)` y recibirlo con `await`.

02 Marco teórico

DEFINICIÓN · ListView y ListView.builder

`ListView` muestra sus hijos en una columna desplazable. Para listas largas o dinámicas se usa `ListView.builder`, que crea los elementos bajo demanda (solo los visibles), lo que es mucho más eficiente.

DEFINICIÓN · GridView

`GridView.builder` organiza los elementos en una cuadrícula desplazable. El `gridDelegate` define cuántas columnas hay y la proporción de cada celda.

DEFINICIÓN · Paso de datos entre pantallas

Hacia adelante: se envían datos a la nueva pantalla por su constructor. **De regreso:** la pantalla devuelve un valor con `Navigator.pop(context, valor)`, y quien navegó lo recibe esperando con `await`.

03 Requisitos previos

Requisito indispensable. Haber completado las Guías N.º 1 a N.º 3 (entorno operativo y manejo básico de widgets y navegación entre dos pantallas).

Conocimientos de apoyo:

Clases y listas en Dart

Widgets de layout

Navigator.push / pop

ADVERTENCIA · Listas dentro de una Column

Un `ListView` o `GridView` colocado dentro de una `Column` provoca el error "Vertical viewport was given unbounded height". En esta guía los usaremos como **body** del Scaffold; si necesitas combinarlos con otros widgets en una `Column`, envuélvelos en `Expanded`.

04 Desarrollo · Parte A — Listas

1 Modelar los datos

1. Crea un proyecto nuevo llamado `catalogo_app`. En `lib/main.dart`, debajo de los imports, define una clase de datos y una lista de ejemplo:

LIB/MAIN.DART · DART

```
class Producto {
  final String nombre;
  final double precio;
  final IconData icono;

  const Producto(this.nombre, this.precio, this.icono);
}

const productos = <Producto>[
  Producto('Teclado', 120.0, Icons.keyboard),
  Producto('Mouse', 60.0, Icons.mouse),
  Producto('Monitor', 850.0, Icons.monitor),
  Producto('Audifonos', 200.0, Icons.headphones),
  Producto('Webcam', 180.0, Icons.videocam),
];
```

NOTA · Modelo de datos

La clase `Producto` agrupa los campos de cada ítem. Trabajar con una lista de objetos —en lugar de textos sueltos— es la base para mostrar y navegar datos reales.

2 Una lista simple con ListView

1. Como primer acercamiento, una `ListView` con hijos fijos. Cada fila es un `ListTile`:

DART · EJEMPLO

```
ListView(
  children: const [
    ListTile(leading: Icon(Icons.keyboard), title: Text('Teclado')),
    ListTile(leading: Icon(Icons.mouse), title: Text('Mouse')),
    ListTile(leading: Icon(Icons.monitor), title: Text('Monitor')),
  ],
)
```

NOTA · ListTile

`ListTile` es una fila estándar con espacios para un icono a la izquierda (`leading`), un título (`title`), un subtítulo (`subtitle`) y más. Ahorra mucho maquetado.

3 Una lista dinámica con ListView.builder

1. Ahora la versión eficiente, que recorre la lista `productos`. Úsala como `body` de la pantalla principal:

LIB/MAIN.DART · DART

```
class PantallaCatalogo extends StatelessWidget {
  const PantallaCatalogo({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Catalogo')),
      body: ListView.builder(
        itemCount: productos.length,
        itemBuilder: (context, index) {
          final p = productos[index];
          return ListTile(
            leading: Icon(p.icono),
            title: Text(p.nombre),
            subtitle: Text('Bs ' + p.precio.toStringAsFixed(2)),
          );
        },
      ),
    );
  }
}
```

EJEMPLO · Cómo funciona el builder

`itemCount` indica cuántos elementos hay; `itemBuilder` se llama por cada índice visible y devuelve el widget de esa fila. Flutter solo construye lo que cabe en pantalla.

05 Desarrollo · Parte B — Cuadrículas

4 Mostrar los datos en una cuadrícula

1. Reemplaza el `body` anterior por una `GridView.builder` de dos columnas, con cada producto en una `Card`:

DART · BODY DEL SCAFFOLD

```
GridView.builder(
  padding: const EdgeInsets.all(12),
  gridDelegate:
    const SliverGridDelegateWithFixedCrossAxisCount(
      crossAxisCount: 2,
      crossAxisSpacing: 12,
      mainAxisSpacing: 12,
    ),
  itemCount: productos.length,
  itemBuilder: (context, index) {
    final p = productos[index];
    return Card(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(p.icono, size: 40),
          const SizedBox(height: 8),
          Text(p.nombre),
        ],
      ),
    );
  },
);
```

NOTA · El `gridDelegate`

`crossAxisCount: 2` fija dos columnas; `crossAxisSpacing` y `mainAxisSpacing` son los espacios entre celdas. `Card` da la tarjeta con sombra y bordes redondeados.

06 Desarrollo · Parte C — Navegación con datos

5 Pasar datos a la pantalla de detalle

1. Crea una pantalla de detalle que reciba un producto por su constructor:

LIB/MAIN.DART · DART

```
class PantallaDetalle extends StatelessWidget {
  final Producto producto;

  const PantallaDetalle({super.key, required this.producto});
```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text(producto.nombre)),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(producto.icono, size: 80),
          const SizedBox(height: 16),
          Text('Bs ' + producto.precio.toStringAsFixed(2)),
        ],
      ),
    ),
  );
}

```

2. En el `ListTile` (o en la `Card`), agrega un `onTap` que navegue enviando el producto:

DART · DENTRO DEL ITEM BUILDER

```

onTap: () {
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (context) => PantallaDetalle(producto: p),
    ),
  );
},

```

6 Recibir un resultado de vuelta

1. En la pantalla de detalle, agrega un botón que devuelva un mensaje al cerrarse:

DART · EN EL BODY DEL DETALLE

```

ElevatedButton(
  onPressed: () => Navigator.pop(
    context, producto.nombre + ' agregado al carrito',
  ),
  child: const Text('Agregar'),
),

```

2. Convierte el `onTap` en asíncrono para `await` el resultado y mostrarlo con un `SnackBar`:

DART · ITEM BUILDER

```

onTap: () async {
  final mensaje = await Navigator.push<String>(
    context,
    MaterialPageRoute(
      builder: (context) => PantallaDetalle(producto: p),
    ),
  );
}

```

```

    ),
  );
  if (mensaje != null && context.mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(mensaje)),
    );
  }
},

```

ADVERTENCIA · context.mounted

Tras un `await` el usuario pudo cerrar la pantalla. Verificar `context.mounted` antes de usar el `context` evita errores cuando el widget ya no está en pantalla.

07 Verificación de resultados y entregable

Marca cada punto como verificación de que completaste la práctica:

- ☐ La lista de productos se muestra con `ListView.builder` y `ListTile`.
- ☐ La versión en cuadrícula (`GridView.builder` + `Card`) funciona y se desplaza.
- ☐ Al tocar un producto se abre la pantalla de detalle con sus datos.
- ☐ El botón "Agregar" regresa a la lista devolviendo un mensaje.
- ☐ El mensaje se muestra en un `SnackBar` al volver.

EJEMPLO · Qué entregar

El archivo `lib/main.dart` terminado y un documento (PDF o Word) con: (1) captura de la lista; (2) captura de la cuadrícula; (3) captura del detalle de un producto; (4) captura del `SnackBar` al volver; y (5) **reto personal**: agrega al menos 3 productos más y un campo nuevo al modelo (por ejemplo, `descripcion`) y muéstralo en el detalle.

08 Cuestionario

1. ¿Qué ventaja tiene `ListView.builder` frente a un `ListView` con children fijos?
2. ¿Para qué sirve el parámetro `itemCount` y qué recibe `itemBuilder`?
3. ¿Qué controla el `gridDelegate` en un `GridView.builder`?
4. Explica cómo se pasan datos hacia adelante y cómo se devuelve un resultado entre pantallas.
5. ¿Por qué se verifica `context.mounted` después de un `await` antes de usar el `context`?

09 Rúbrica de evaluación

Criterio	Puntaje
Lista dinámica con <code>ListView.builder</code>	15 %
Cuadrícula con <code>GridView.builder</code> y <code>Card</code>	15 %

Paso de datos a la pantalla de detalle	20 %
Resultado de vuelta con SnackBar	20 %
Reto personal y capturas	20 %
Cuestionario resuelto	10 %

10 Referencias

- Flutter — Work with long lists (ListView.builder). docs.flutter.dev/cookbook/lists/long-lists
- Flutter — Create a grid list (GridView). docs.flutter.dev/cookbook/lists/grid-lists
- Flutter — Send data to a new screen. docs.flutter.dev/cookbook/navigation/passing-data
- Flutter — Return data from a screen. docs.flutter.dev/cookbook/navigation/returning-data

Contenido basado en la documentación oficial de Flutter (v3.44). Guía elaborada para INF 662 — UATF.